

Análisis de Lenguajes de Programación

Sintaxis

10 de Agosto de 2026

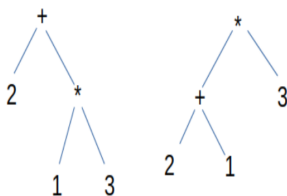
- ▶ La sintaxis describe la forma que van a tener los programas válidos del lenguaje.
- ▶ **Sintaxis concreta:**
 - ▶ modela las **secuencias de caracteres** que son aceptadas como programas sintácticamente válidos
 - ▶ incluye información sobre la representación. Por ejemplo: cómo está parentizada una expresión, cómo se separan los elementos de una lista, si los operadores son prefijos o infijos, etc.
- ▶ **Sintaxis abstracta:**
 - ▶ modela la **estructural** esencial de los programas sintácticamente válidos
 - ▶ es independiente de cualquier representación
 - ▶ son simples, ya que no se consideran detalles de la notación concreta

Sintaxis abstracta: Ejemplo

$exp ::= n \mid exp + exp \mid exp * exp$

donde $n \in \mathbb{N}$

Las siguientes son árboles sintácticos:



que corresponden a las expresiones $2 + (1 * 3)$ y $(2 + 1) * 3$

Sintaxis concreta: Ejemplo

$$\text{exp} ::= n \mid \text{exp} \text{'+'} \text{exp} \mid \text{exp} \text{'*'} \text{exp} \mid \text{'(' exp ') '}$$
$$n ::= d \mid dn$$
$$d ::= \text{'0'} \mid \text{'1'} \mid \dots \mid \text{'9'}$$

Algunas derivaciones:

$$\text{exp} \rightarrow n \rightarrow dn \rightarrow dd \rightarrow 1d \rightarrow 12$$
$$\begin{aligned} \text{exp} &\rightarrow \text{exp} \text{'+'} \text{exp} \rightarrow \text{exp} \text{'+'} \text{exp} \text{'*'} \text{exp} \rightarrow \\ \text{exp} \text{'+'} \text{exp} \text{'*'} n &\rightarrow \dots \rightarrow 2 \text{'+'} 1 \text{'*'} 3 \end{aligned}$$
$$\begin{aligned} \text{exp} &\rightarrow \text{exp} \text{'*'} \text{exp} \rightarrow \text{exp} \text{'+'} \text{exp} \text{'*'} \text{exp} \rightarrow \\ \text{exp} \text{'+'} \text{exp} \text{'*'} n &\rightarrow \dots \rightarrow 2 \text{'+'} 1 \text{'*'} 3 \end{aligned}$$

Desambiguamos la gramática

1. Definimos convenciones sobre la precedencia y la asociatividad de los operadores.
 2. Reflejamos las convenciones en la gramática.
- Reflejamos que el producto tiene más precedencia que la suma:

```
exp ::= exp '+' exp | term
term ::= term '*' term | atom
atom ::= '(' exp ') ' | n
```

- Reflejamos además que ambos operadores asocian a izquierda:

```
exp ::= exp '+' term | term
term ::= term '*' atom | atom
atom ::= '(' exp ') ' | n
```

Gramáticas Libres de contexto

- ▶ Una forma de definir la sintaxis de un lenguaje es mediante una gramática Libre de contexto (CFG).
- ▶ La mayoría de los lenguajes de programación definen su sintaxis mediante una CFG.
- ▶ Un **lenguaje es libre de contexto** si hay una CFG que lo genera.
- ▶ Capturan ciertas nociones que permiten desambiguar los lenguajes:
 - ▶ paréntesis balanceados
 - ▶ palabras claves emparejadas (como begin y end)

Una CFG puede ser definida por una 4-tupla (N, T, P, S) , donde:

- ▶ N es un conjunto finito de **no terminales**.
- ▶ T es un conjunto finito de **terminales**, donde $N \cap T = \emptyset$.
- ▶ P es un conjunto de **producciones** de la forma $A \rightarrow \alpha$, donde $A \in N$ y $\alpha \in (N \cup T)^*$, siendo $*$ el operador estrella de Kleene.
- ▶ S es un **símbolo inicial** que pertenece a N .

- ▶ $G = (S, \{a, b\}, P, S)$, donde P :

$$S \rightarrow aSb$$

$$S \rightarrow \epsilon$$

Esta gramática genera el lenguaje: $\{a^n b^n : n \geq 0\}$

- ▶ Las producciones con el mismo lado izquierdo se pueden agrupar (notación de Backus-Naur o BNF):

$$S \rightarrow aSb \mid \epsilon$$

- ▶ También se utiliza $::=$ en lugar de $\rightarrow$ en las producciones.

Relación de derivación

- ▶ Sea (N, T, P, S) una gramática definimos la relación binaria $\Rightarrow$ sobre $(N \cup T)^*$, como la relación tal que:

$$\alpha A \gamma \Rightarrow \alpha B \gamma$$

donde $A \rightarrow B$ es una producción de G .

- ▶ Y la relación de derivación $\Rightarrow^*$, como la clausura reflexiva transitiva de $\Rightarrow$. Es decir, es la menor relación sobre $(N \cup T)^*$ tal que:
 - ▶ Si $\alpha \Rightarrow \beta$, entonces $\alpha \Rightarrow^* \beta$
 - ▶ $\alpha \Rightarrow^* \alpha$
 - ▶ Si $\alpha \Rightarrow^* \beta$ y $\beta \Rightarrow^* \gamma$, entonces $\alpha \Rightarrow^* \gamma$
- ▶ Lenguaje generado por G :

$$L(G) = \{w \mid w \in T^* \wedge S \Rightarrow^* w\}$$

Ejemplo

$$S \Rightarrow aSb$$

$$S \Rightarrow \epsilon$$

$$aSb \Rightarrow aaSbb$$

$$aaS \Rightarrow aaaSb$$

$$S \Rightarrow^* aSb$$

$$S \Rightarrow^* \epsilon$$

$$aSb \Rightarrow^* aaSbb$$

$$aaSbb \Rightarrow^* aabb$$

$$S \Rightarrow^* aaabbb$$

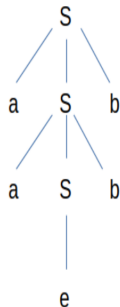
$$L(G) = \{a^n b^n : n \geq 0\}$$

- ▶ Los lenguajes libre de contexto pueden definirse también a través de autómatas no deterministas.
- ▶ La unión de dos lenguajes libres de contexto es también libre de contexto. (La intersección, no necesariamente.)
- ▶ Determinar si dos CFGs generan el mismo lenguaje no es decidable.

Árbol de parseo

Dada una gramática (N, T, P, S) , $S \Rightarrow \alpha$ sii α es el **resultado** de un árbol de parseo, siendo éste la cadena que se forma al unir las etiquetas de las hojas del árbol (de izquierda a derecha).

Para la CFG del ejemplo, la cadena $aabb$ tiene el siguiente árbol:



Árbol de parseo: definición

Un árbol es una derivación o **árbol de parseo** de una CFG $G = (N, T, P, S)$ si:

- ▶ cada nodo tiene una etiqueta en $N \cup T \cup \{\epsilon\}$,
- ▶ la raíz tiene etiqueta S ,
- ▶ las etiquetas de los nodos interiores están en N ,
- ▶ Si un nodo con etiqueta A tiene k hijos con etiquetas $X_1, \dots, X_k$, entonces $A \rightarrow X_1, \dots, X_k$ en una regla en P .
- ▶ Si un nodo tiene etiqueta ϵ , entonces es una hoja y es único hijo.

Gramáticas ambiguas

- ▶ Una CFG G es **ambigua** si una palabra en $L(G)$ tiene más de un árbol de parseo.
- ▶ En general, los lenguajes libres de contextos pueden ser generados por gramáticas ambiguas y no ambiguas. Pero existen algunos que sólo pueden ser generados por CFG ambiguas, estos son llamados **lenguajes inherentemente ambiguos**.
- ▶ Determinar si una CFG es ambigua no es decidible.

Desambiguando gramáticas

- ▶ Resolver la ambigüedad es importante, por ejemplo dos árboles de parseos pueden tener distinta semántica.
- ▶ En el ejemplo introductorio vimos como desambiguar una gramática fijando reglas de precedencia y asociatividad entre los operadores.
- ▶ Veremos cómo solucionar de manera similar, agregando reglas y modificando la gramática, el problema de ambigüedad que se conoce como "**else colgado**".

Desambiguando gramática: else colgado

Supongamos una gramática con la siguiente producción:

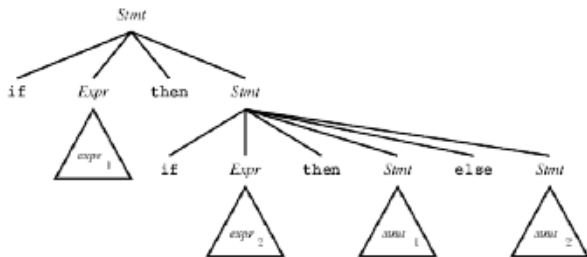
$$stm \rightarrow \text{if } exp \text{ then } stm \mid \text{if } exp \text{ then } stm \text{ else } stm$$

El siguiente programa tiene dos árboles de parseo:

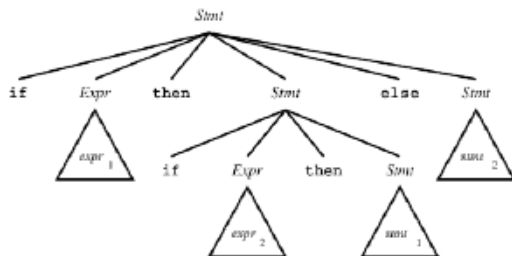
$$\text{if } exp_1 \text{ then if } exp_2 \text{ then } stm_1 \text{ else } stm_2$$

Desambiguando gramática: else colgado

Árbol 1:



Árbol 2:



Desambiguando gramática: else colgado

- ▶ En el primer árbol el '**else**' está asociado al '**then**' más cercano, mientras que en el segundo árbol está asociado al primer '**then**'.
- ▶ Agregamos la siguiente regla: "cada '**else**' está asociado al '**then**' más cercano que no está asociado a otro '**else**'"
- ▶ Transformamos la gramática a una equivalente pero **sin ambigüedades**.

Desambiguando gramática: else colgado

La sentencia que está entre '**then**' y '**else**' (matched) no puede contener un '**if_then**', ya que esto violaría la regla.

$$stm \rightarrow matched \mid unMatched$$
$$matched \rightarrow \text{if } exp \text{ then } matched \text{ else } matched \mid \dots$$
$$unMatched \rightarrow \text{if } exp \text{ then } stm \mid \\ \text{if } exp \text{ then } matched \text{ else } unMatched$$

Ejercicio

Probar que la siguiente gramática es ambigua y desambiguarla.

$$\begin{aligned} \text{exp} &::= v \mid \text{exp } '+' \text{exp} \mid \text{exp } '-' \text{exp} \mid \text{exp } '*' \text{exp} \mid '(' \text{exp} ')', \\ v &::= x \mid y \mid z \end{aligned}$$

Sintaxis abstracta

- ▶ Vimos que al desambiguar una gramática concreta obtuvimos otra más difícil de interpretar.
- ▶ La sintaxis abstracta no tiene problemas de ambigüedad, ya que define árboles.
- ▶ Llamaremos **árbol de sintaxis abstracta** (AST), a la representación como árbol de la estructura sintáctica de un lenguaje.

Por ejemplo, el AST en BNF del lenguaje del ejercicio es:

$$\textit{exp} \rightarrow n \mid \textit{exp} + \textit{exp} \mid \textit{exp} - \textit{exp} \mid \textit{exp} * \textit{exp}$$

- ▶ Las gramáticas son más simples.

Implementación de AST

Podemos implementar el AST de un lenguaje en Haskell con un tipo de datos algebraico. Por ejemplo:

```
data Exp =    Num Int
            | Sum Exp Exp
            | Prod Exp Exp
            | Minus Exp Exp
```

En la implementación notamos que los elementos de `Exp` son **árboles** y los nombres de los constructores son arbitrarios.

Lenguaje de booleanos y naturales

Definimos el AST del lenguaje como:

t	$\rightarrow$	true
		false
		if t then t else t
		0
		succ t
		pred t
		iszero t

- ▶ t es una **metavariable** (variable que pertenece al metalenguaje) y es usada para representar un término del lenguaje objeto (el cual se describe).
- ▶ usaremos también letras cercanas a t (u, v, t_1, t') en el lado derecho de las reglas.

Conjunto de términos

- ▶ Los AST definen el **conjunto de términos** del lenguaje.
- ▶ Una forma alternativa de definir el conjunto de términos de un lenguaje es mediante una definición inductiva.
Por ejemplo, definimos T como el menor conjunto tal que:
 - ▶ $\{\mathbf{true}, \mathbf{false}, \mathbf{0}\} \subseteq T$
 - ▶ Si $t \in T$ entonces $\{\mathbf{suc } t, \mathbf{pred } t, \mathbf{iszero } t\} \subseteq T$
 - ▶ Si $t, u, v \in T$ entonces $\mathbf{if } t \mathbf{ then } u \mathbf{ else } v \in T$

Conjunto de términos

Otra forma de definir el conjunto de términos (más concreta), es mediante un procedimiento que genera los elementos del conjunto. Por ejemplo, daremos una definición alternativa de T .

- Primero damos una definición de S_i :

$$\begin{aligned} S_0 &= \emptyset \\ S_{i+1} &= \{\mathbf{true}, \mathbf{false}, \mathbf{0}\} \\ &\quad \cup \{\mathbf{suc } t, \mathbf{pred } t, \mathbf{iszero } t \mid t \in S_i\} \\ &\quad \cup \{\mathbf{if } t \mathbf{ then } u \mathbf{ else } v \mid t, u, v \in S_i\} \end{aligned}$$

- Luego definimos:

$$S = \bigcup_{i \in \mathbb{N}} S_i$$

Prueba de que $T = S$

Dado que T fue definido como el menor conjunto que satisface ciertas condiciones, para probar $T = S$, basta con probar:

1. S satisface las condiciones de T ,
2. cualquier conjunto que satisfaga las condiciones contiene a S (S es el menor conjunto que satisface las condiciones).

- ▶ Los lenguajes de programación son lenguajes formales.
- ▶ La **sintaxis concreta** modela a los programas como secuencias de caracteres, mientras que la **sintaxis abstracta** como árboles.
- ▶ Usamos **gramáticas libres de contexto** para describir la sintaxis concreta de lenguajes.
- ▶ La sintaxis concreta suele ser ambigua, mientras que la abstracta no.
- ▶ Para desambiguar gramáticas definimos reglas (de precedencia, asociatividad, etc) y las reflejamos en la gramática.
- ▶ Definimos la sintaxis abstracta mediante un árbol de sintaxis abstracta (AST).
- ▶ Implementamos el AST de un lenguaje en Haskell con un tipo algebraico.
- ▶ El AST es un conjunto de términos que puede definirse inductivamente.